

Artificial Intelligence - Report Lab 2

Anh Khoi Pham - ITCSIU23018

April 11, 2026

1 The Algorithm

BFS

```
1  def bfs(self, return_home=False):
2      start = (self.state.pos_x, self.state.pos_y)
3      frontier = deque()
4      frontier.append((start, []))
5
6      explored = set()
7
8      while frontier:
9          node, path = frontier.popleft()
10
11         if node in explored:
12             continue
13         explored.add(node)
14
15         x, y = node
16
17         if return_home:
18             if node == self.home_pos:
19                 self.route = path.copy()
20                 return
21         else:
22             if self.state.world[x][y] ==
AGENT_STATE_UNKNOWN:
23                 self.route = path.copy()
24                 return
25
26         for dx, dy in [(1, 0), (0, 1), (-1, 0),
(0, -1)]:
27             nx, ny = x + dx, y + dy
28
29             # check boundary
30             if not (0 <= nx < self.state.
world_width and 0 <= ny < self.
state.world_height):
31                 continue
32             # check wall
33             if self.state.world[nx][ny] ==
AGENT_STATE_WALL:
```

```
34         continue
35
36         child = (nx, ny)
37         if child in explored or any(child ==
38             n for n, _ in frontier):
39             continue
40
41         frontier.append((child, path + [child]))
```

How it works? The algorithm works as follows. While the `frontier` queue is not empty, it removes the first node together with the path leading to it. Next, it checks whether the map has already been fully explored and whether the agent is returning home. If the current node is still marked as "unknown", the agent still has unexplored space to visit, so the `route` is updated with the path to that node. Otherwise, the algorithm expands the current node by adding its valid children to the `frontier` according to the BFS strategy.

DFS

```
1  def dfs(self, return_home=False):
2      start = (self.state.pos_x, self.state.pos_y)
3      frontier = deque()
4      frontier.append((start, []))
5
6      explored = set()
7
8      while frontier:
9          node, path = frontier.pop()
10
11         if node in explored:
12             continue
13         explored.add(node)
14
15         x, y = node
16
17         if return_home:
18             if node == self.home_pos:
19                 self.route = path.copy()
```

```
20         return
21     else:
22         if self.state.world[x][y] ==
23             AGENT_STATE_UNKNOWN:
24             self.route = path.copy()
25             return
26     for dx, dy in [(1, 0), (0, 1), (-1, 0),
27                   (0, -1)]:
28         nx, ny = x + dx, y + dy
29
30         # check boundary
31         if not (0 <= nx < self.state.
32             world_width and 0 <= ny < self.
33             state.world_height):
34             continue
35
36         # check wall
37         if self.state.world[nx][ny] ==
38             AGENT_STATE_WALL:
39             continue
40
41         child = (nx, ny)
42         if child in explored or any(child ==
43             n for n, _ in frontier):
44             continue
45
46         frontier.append((child, path + [child
47             ]))
```

How it works Similar logic with the BFS. However, instead of using queue (LIFO) like BFS, DFS use stack (FIFO) for the frontier.

2 Comparison

Table 1: Comparison of BFS and DFS performance in different environment sizes.

Env. Size	Breadth-First Search			Depth-First Search		
	Nodes	Len.	Optimal?	Nodes	Len.	Optimal?
	Explored		Score?	Explored		Score?
5 × 5	9	48	Optimal / 46	9	128	Non-optimal / -36
10 × 10	64	192	Optimal / 504	64	1227	Non-optimal / -731
15 × 15	169	388	Optimal / 1623	169	5600	Non-optimal / -3993
20 × 20	324	652	Optimal / 2571	324	14928	Non-optimal / -11301

BFS is optimal because it explores nodes layer-by-layer, ensuring the first path found to the goal is the shortest. In contrast, DFS is not optimal as it explores as deep as possible along one branch, often resulting in a much longer solution length than necessary.